

Student Full Name	Kolganatov Ramazan
Student ID Number	230103182
Institutional Email / Git URL	230103182@sdu.edu.kz
Practicum Date & Time (Slot)	25.09.2025
Laptop Model & OS	Dell G16 7630, Windows 11 Home 25H2 (x64)
CPU Model & Generation	Intel Core i9-13900HX, 13th Generation
Physical Cores (P/E breakdown if hybrid)	24 cores (8 P-cores + 16 E-cores)
Logical SMT Threads (omp_get_max_threads)	32
Target Workload Size N	13,182,000
L1 / L2 Cache Line Size	64 bytes

Phase 1

```
PS C:\Users\ramaz\Downloads\labs> type hw_info.txt

Name           : 13th Gen Intel(R) Core(TM) i9-13900HX
NumberOfCores   : 24
NumberOfLogicalProcessors : 32
```

```
PS C:\Users\ramaz\Downloads\labs> python hardware_info.py
=== Hardware Information ===
OS: Windows 11
CPU: Intel64 Family 6 Model 183 Stepping 1, GenuineIntel
Physical Cores: 24
Logical Cores: 32
Python CPU Count: 32
```

```
PS C:\Users\ramaz\Downloads\labs> python collatz_test.py
1: 0 steps
2: 1 steps
3: 7 steps
4: 2 steps
5: 5 steps
6: 8 steps
7: 16 steps
8: 3 steps
9: 19 steps
10: 6 steps
11: 14 steps
12: 9 steps
13: 9 steps
14: 17 steps
15: 17 steps
16: 4 steps
17: 12 steps
18: 20 steps
19: 20 steps
20: 7 steps
21: 7 steps
22: 15 steps
23: 15 steps
24: 10 steps
25: 23 steps
26: 10 steps
27: 111 steps
```

Phase 2

1 Run

```
PS C:\Users\ramaz\Downloads\labs> python collatz_seq.py
N = 13182000
Starting sequential benchmark...
Time: 122.661240 seconds
Maximum stopping time: 688
Number: 11200681
Checksum: 84256261
```

2 Run

```
PS C:\Users\ramaz\Downloads\labs> python collatz_seq.py
N = 13182000
Starting sequential benchmark...
Time: 124.092114 seconds
Maximum stopping time: 688
Number: 11200681
Checksum: 84256261
```

3 Run

```
PS C:\Users\ramaz\Downloads\labs> python collatz_seq.py
N = 13182000
Starting sequential benchmark...
Time: 111.300828 seconds
Maximum stopping time: 688
Number: 11200681
Checksum: 84256261
```

N = 13,182,000

Run 1 (warmup): 122.661240 s

Run 2: 124.092114 s

Run 3: 111.300828 s

T_seq = 117.696471 s

Maximum stopping time = 688

Number = 11,200,681

Checksum = 84,256,261

Phase 3

k	T_k (s)	S_emp(k)	S_theo(k)	$\Delta = \text{Theo} - \text{Emp}$
1	132.26694 1	0.88984 0	1.000000	0.110160
2	107.15789 2	1.09834 6	1.098346	0.000000
4	58.925025	1.99739 4	1.155149	-0.842245
8	38.099842	3.08915 9	1.185811	-1.903348
16	24.276325	4.84820 0	1.201761	-3.646438

Phase 4

```
N = 13182000
Processes = 24

Variant 1: Naive
Time: 21.661424 seconds
Hits (>100 steps): 10623640
Maximum stopping time: 688
Number: 11200681

Variant 2: Mitigated
Time: 21.146345 seconds
Hits (>100 steps): 10623640
Maximum stopping time: 688
Number: 11200681
```

Experiment A — Python multiprocessing approximation

Processes: 24

Naive:

Execution time = 21.661424 s

Hits (>100 steps) = 10,623,640

Mitigated:

Execution time = 21.146345 s

Hits (>100 steps) = 10,623,640

Both variants produced the same computational result.

Experiment B

Scheduling	Chunk	Time (s)	Throughput (iter/s)
static	N / 24	18.337730	718,845.79
static, 1000	1,000	14.158547	931,027.73
dynamic, 100	100	14.040164	938,877.92
dynamic, 10000	10,000	14.094850	935,235.20
guided	adaptive	22.016359	598,736.60

The static schedule was slower (18.34 s) than the chunked schedules. dynamic(100) achieved 14.04 s, while dynamic(10000) achieved 14.09 s. Their execution times were almost identical, suggesting that in this workload the benefit of finer-grained load balancing was small compared with the scheduling overhead. The guided implementation was slower at 22.02 s. Therefore, in this Python multiprocessing experiment, very small chunks did not provide a substantial advantage over larger chunks.

Phase 5

Q1. False Sharing

The cache line size of the CPU is 64 bytes. In the naive OpenMP implementation, each thread updates its own element of the `hit_count` array. However, several neighboring int elements can fit into the same 64-byte cache line.

The problem is that cache coherence protocols such as MESI/MOESI operate at the cache-line level rather than at the individual integer level. When one thread modifies its counter, the cache line may be invalidated in another core's cache. This causes the cache line to move between cores and creates additional coherence and memory traffic.

Therefore, even though the threads are updating different array elements, they can interfere with each other because their counters are located on the same cache line. This is known as false sharing.

Q2. SMT Scaling

My CPU has 24 physical cores and 32 logical threads. The measurements performed for this experiment used thread counts up to 16, so the collected results do not empirically show what happens between 24 physical cores and 32 logical threads.

SMT can improve throughput because two logical threads can share the execution resources of one physical core. However, the two threads also share resources such as execution units, cache capacity and memory bandwidth. Therefore, increasing the number of logical threads does not necessarily produce proportional speedup.

On a hybrid CPU such as the Intel Core i9-13900HX, the physical cores also have different characteristics because the processor contains P-cores and E-cores. Consequently, thread placement and shared hardware resources can affect the scaling behavior.

Q3. Amdahl's Law vs. Empirical Speedup

The theoretical Amdahl speedup at 16 threads is approximately 1.20, while the measured empirical speedup was approximately 4.85. Therefore, the simple Amdahl model does not accurately describe the measured scaling.

One reason is that the model assumes a fixed serial fraction and does not capture all implementation and hardware effects. Two important physical factors are:

Cache and memory hierarchy effects — cache locality, cache capacity and memory bandwidth can change as the number of workers increases.

Shared and heterogeneous CPU resources — SMT threads share physical-core resources, while the CPU contains both P-cores and E-cores with different performance characteristics.

In addition, the experiment used Python multiprocessing rather than the required OpenMP implementation, so the measured parallel timings should not be interpreted as a direct OpenMP hardware result.

Q4. Scheduling

The measured results were:

Schedule	Time (s)
static	18.33773 0
static,1000	14.15854 7
dynamic,100	14.04016 4
dynamic,10000	14.09485 0
guided	22.01635 9

In this experiment, dynamic,100 was slightly faster than dynamic,10000. The difference was only about 0.055 seconds, so the measurements do not provide strong evidence of a clear point where OpenMP lock-queue contention outweighed the benefits of load balancing.

The Python multiprocessing experiment cannot directly measure OpenMP's scheduling lock/queue contention. Therefore, the exact chunk size at which lock contention becomes more expensive than load balancing cannot be determined reliably from these measurements.

Phase 6

Hardware:

CPU: Intel Core i9-13900HX

Physical cores: 24

Logical threads: 32

Cache line size: 64 bytes

Workload N: 13182000

Sequential baseline:

Warm-up Run 1: 122.661240 s

Run 2: 124.092114 s

Run 3: 111.300828 s

$T_{\text{seq}} = (\text{Run 2} + \text{Run 3}) / 2 = 117.696471 \text{ s}$

Correctness:

Maximum stopping time: 688

Number: 11200681

Checksum: 84256261

Scaling:

k=1: 132.266941 s, speedup 0.889840

k=2: 107.157892 s, speedup 1.098346

k=4: 58.925025 s, speedup 1.997394

k=8: 38.099842 s, speedup 3.089159

k=16: 24.276325 s, speedup 4.848200

Using the two-worker empirical speedup, the estimated parallel fraction is:

$p = 2 * (1 - 1/S_{\text{emp}}(2)) = 0.179081 \text{ (17.91\%)}$.

The Amdahl model predicts approximately 1.20x speedup at 16 workers, while the measured empirical speedup is approximately 4.85x. This large difference shows that the simple fitted Amdahl model does not describe the measured Python multiprocessing implementation accurately.

Possible factors include cache/memory-hierarchy effects, heterogeneous P-core/E-core resources, SMT resource sharing, and differences between the sequential and multiprocessing runtime implementations.

Important methodological note:

The measurements in this report were performed using Python multiprocessing rather than the OpenMP implementation specified by the worksheet. Therefore, these results should be treated as empirical Python multiprocessing measurements, not as direct OpenMP measurements.

Scheduling results:

static = 18.337730 s

static,1000 = 14.158547 s

dynamic,100 = 14.040164 s

dynamic,10000 = 14.094850 s

guided = 22.016359 s

Dynamic(100) was slightly faster than dynamic(10000), with a difference of approximately 0.055 s. The Python experiment does not directly measure OpenMP lock-queue contention, so no reliable OpenMP contention crossover point can be determined from these measurements.

False-sharing approximation:

naive = 21.661424 s

mitigated = 21.146345 s

difference = 0.515079 s

This Python multiprocessing experiment is only an approximation and does not directly demonstrate OpenMP false sharing because Python processes do not use the same shared integer array as OpenMP threads.